

ASEN 5264 - DMU

Homework #4

Bijan Jourabchi

Problem 1:

- a) After a large number of pulls, it is reasonable to assume that $\rho_a \rightarrow \theta_a$. However, since we are using an ϵ -greedy policy, we still choose randomly with probability ϵ . With probability $1 - \epsilon$ we will choose greedily, always picking arm 3. With $\epsilon = 0.15$, we can solve for the probability of pulling arm a .

$$P(a_1) = \epsilon/3 = 0.05$$

$$P(a_2) = \epsilon/3 = 0.05$$

$$P(a_3) = \epsilon/3 + 1 - \epsilon = 0.9$$

The expected payoff per pull is given as $\mathbb{E}[R]$, where $\mathbb{E}[R]$ is given by:

$$\mathbb{E}[R] = \sum_{k=1}^3 P(R_i)P(a_i) = 0.2 * 0.05 + 0.3 * 0.05 + 0.7 * 0.9 = \boxed{0.655}$$

- b) Again, after a large number of pulls, we have $\rho_a \rightarrow \theta_a$. A softmax policy tells us $P(a_3) \propto e^{\lambda \rho_3} = e^{5\theta_3}$. In order to obtain a valid probability, we must multiply by a normalizing constant $\frac{1}{c}$ where $c = \sum_{k=1}^3 e^{5\theta_k}$.

$$P(a_3) = \frac{e^{5\theta_3}}{\sum_{k=1}^3 e^{5\theta_k}} = \frac{33.11}{40.32} = \boxed{0.821}$$

- c) Starting with a prior distribution Beta(1,1), we update the distribution for each arm with Beta(1 + w , 1 + l) where w and l are the total number of wins and losses respectively. For each arm we have the following beliefs.

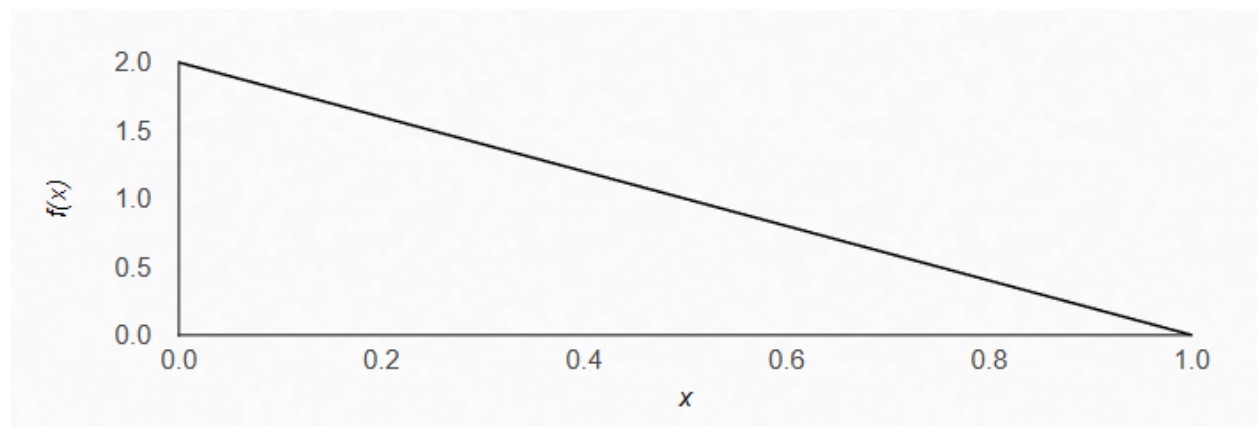


Figure 1: Belief for arm 1

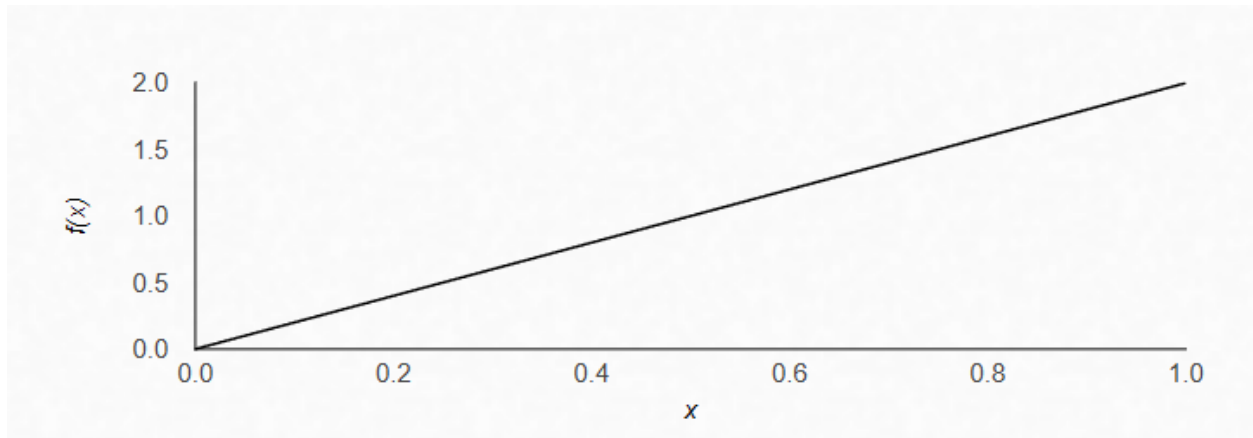


Figure 2: Belief for arm 2

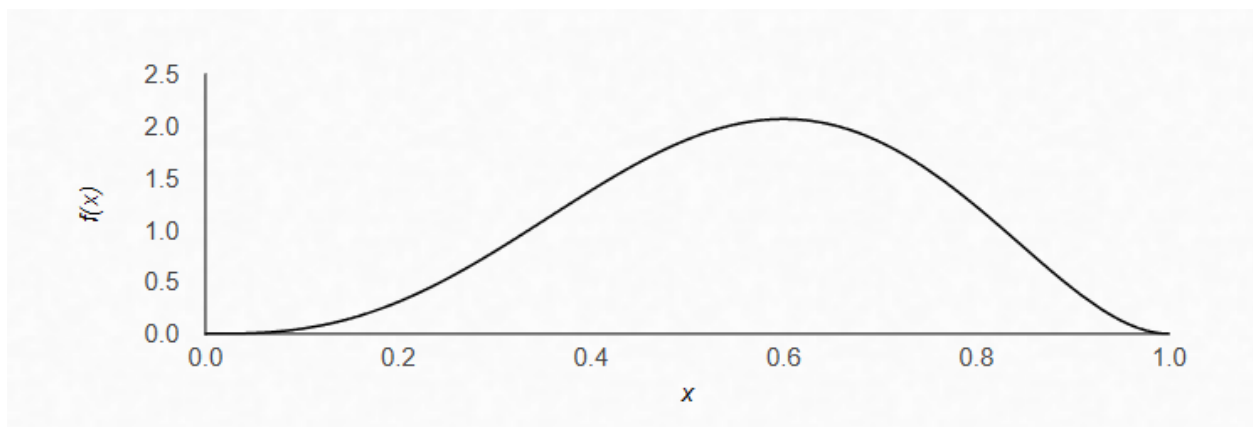


Figure 3: Belief for arm 3

- d) First, we would sample a value $\hat{\theta}_a$ from each belief distribution, then we would choose the arm which had the largest $\hat{\theta}_a$, that is $\arg \max_a \hat{\theta}_a$. Some theoretical values that could be sampled are $\hat{\theta}_1 = 0.1$, $\hat{\theta}_2 = 0.8$, $\hat{\theta}_3 = 0.65$. In this case, Thompson sampling would choose arm 2, observe the outcome and update the belief for arm 2.

Problem 2:

Given a policy parameterized with $\theta = [\theta_L, \theta_R]$, we can take the logarithm of the policy, $\log(\pi_\theta(s | a))$ giving us,

$$\begin{aligned}\log(\pi_\theta(a | s)) &= \log\left(\frac{e^{\theta_a}}{e^{\theta_L} + e^{\theta_R}}\right) = \log(e^{\theta_a}) - \log(e^{\theta_L} + e^{\theta_R}) \\ \log(\pi_\theta(a | s)) &= \theta_a - \log(e^{\theta_L} + e^{\theta_R})\end{aligned}$$

Using this expression, we can calculate the policy gradients for the given trajectories.

- a) Given $\tau = (1, L, 10, 2)$, the log policy is $\log(\pi_\theta(L | 1)) = \theta_L - \log(e^{\theta_L} + e^{\theta_R})$. The gradient of the log policy is $\nabla_\theta \log(\pi_\theta(L | 1))$ where $\nabla_\theta = [\frac{\partial}{\partial \theta_L} \quad \frac{\partial}{\partial \theta_R}]^T$. Evaluating the gradient at $\theta = [0.5, 0.5]$ we get

$$\begin{aligned}\left[\frac{\partial \log(\pi_\theta(L | 1))}{\partial \theta_L}\right]_{0.5, 0.5} &= \left[1 - \frac{e^{\theta_L}}{e^{\theta_L} + e^{\theta_R}}\right]_{0.5, 0.5} = 1 - \frac{e^{0.5}}{2e^{0.5}} = \frac{1}{2} \\ \left[\frac{\partial \log(\pi_\theta(L | 1))}{\partial \theta_R}\right]_{0.5, 0.5} &= \left[-\frac{e^{\theta_R}}{e^{\theta_L} + e^{\theta_R}}\right]_{0.5, 0.5} = -\frac{e^{0.5}}{2e^{0.5}} = -\frac{1}{2}\end{aligned}$$

Thus,

$$\nabla_\theta \log(\pi_\theta(a | s)) = \begin{bmatrix} 1/2 \\ -1/2 \end{bmatrix}$$

- b) Given $\tau = (1, R, 20, 2)$, the log policy becomes $\log(\pi_\theta(R | 1)) = \theta_R - \log(e^{\theta_L} + e^{\theta_R})$.

$$\begin{aligned}\left[\frac{\partial \log(\pi_\theta(R | 1))}{\partial \theta_L}\right]_{0.5, 0.5} &= \left[-\frac{e^{\theta_R}}{e^{\theta_L} + e^{\theta_R}}\right]_{0.5, 0.5} = -\frac{e^{0.5}}{2e^{0.5}} = -\frac{1}{2} \\ \left[\frac{\partial \log(\pi_\theta(R | 1))}{\partial \theta_R}\right]_{0.5, 0.5} &= \left[1 - \frac{e^{\theta_R}}{e^{\theta_L} + e^{\theta_R}}\right]_{0.5, 0.5} = 1 - \frac{e^{0.5}}{2e^{0.5}} = \frac{1}{2}\end{aligned}$$

Thus,

$$\nabla_\theta \log(\pi_\theta(a | s)) = \begin{bmatrix} -1/2 \\ 1/2 \end{bmatrix}$$

Problem 3:

Using Flux.jl to fit a neural network to approximate the function $f(x) = (1-x) \sin(20 \log(x + 0.2))$, I got the results seen in figure 4.

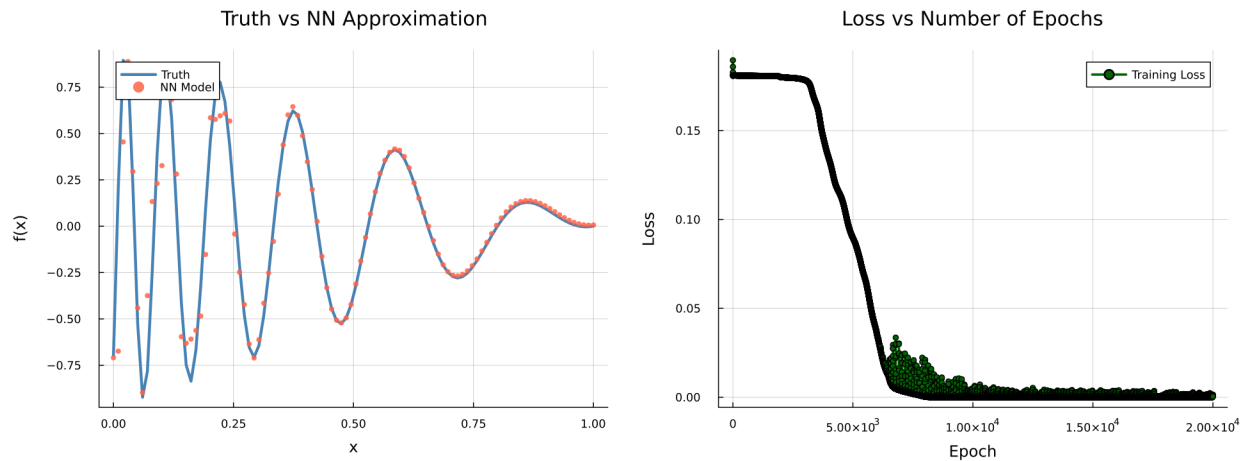


Figure 4: Neural Network Function Approximation

3.1 Code For Problem 3

```

1  using Flux
2  using Flux.Losses: mse
3  using Plots
4  using Statistics: mean
5  using Plots.PlotMeasures: mm
6  using ProgressMeter
7  using Random
8
9  f = x -> (1 - x) * sin(20 * log(x + 0.2))
10 #f = x -> sin(x)
11
12 function make_training_data(f)
13     x = collect(range(0,1,length=35))
14     vcat(x, 0.001)
15     y = f.(x)
16     return x, y
17 end
18
19 function lossfxn(m, x, y)
20     return mse(m(x), y)
21 end
22
23 function train(x_data, y_data; learning_rate=1e-3, n_epochs=1_000,
24               save_every=50, minibatch_size=1)
25     x_data = ndims(x_data) == 1 ? reshape(Float32.(x_data), 1, :) :
26             Float32.(x_data)
27     y_data = ndims(y_data) == 1 ? reshape(Float32.(y_data), 1, :) :
28             Float32.(y_data)

```

```

26 @assert size(x_data, 2) == size(y_data, 2) "x_data_and_y_data_must_
    have_the_same_number_of_samples"
27 @assert minibatch_size >= 1 "minibatch_size_must_be_>=1"
28
29 model = Chain(
30     Dense(1=>32, tanh),
31     Dense(32=>64, tanh),
32     Dense(64=>32, tanh),
33     Dense(32=>1)
34 )
35 opt_state = Flux.setup(Adam(learning_rate), model)
36
37 losses = Float32[]
38 models = [deepcopy(model)]
39
40
41 n_minibatches = Int(length(y_data) / minibatch_size)
42
43 @showprogress for epoch in 1:n_epochs
44     batch_loss = zero(Float32)
45
46     for i in 1:n_minibatches
47         start_idx = (i - 1) * minibatch_size + 1
48         end_idx = i * minibatch_size
49         idxs = start_idx:end_idx
50         x_minibatch = x_data[:, idxs]
51         y_minibatch = y_data[:, idxs]
52
53         function minibatch_objective(model)
54             return lossfn(model, x_minibatch, y_minibatch)
55         end
56
57         minibatch_loss, grads = Flux.withgradient(minibatch_objective,
58             model)
59
60         Flux.update!(opt_state, model, grads[1])
61
62         batch_loss += minibatch_loss / n_minibatches
63     end
64
65     push!(losses, batch_loss)
66
67     if epoch % save_every == 0
68         push!(models, deepcopy(model))
69     end
70
71     return models, losses
72 end
73
74 x, y = make_training_data(f)
75
76 training_plot = plot(
77     vec(x), vec(y),

```

```

78     label="Training_Data",
79     xlabel="x",
80     ylabel="f(x)",
81     title="Training_Set",
82     linewidth=1.8,
83     marker=:circle,
84     markersize=2.5,
85     color=:navy,
86     legend=:topright,
87     gridalpha=0.25
88 )
89 savefig("training_set.png")
90
91 models, losses = train(x, y; learning_rate=5e-4, n_epochs=20_000,
    minibatch_size=5)
92
93 xs = collect(range(0.0f0, 1.0f0, length=100))
94 truth_vals = f.(xs)
95 pred_vals = [models[end](Float32[x])[1] for x in xs]
96
97 p1 = plot(
98     xs, truth_vals,
99     label="Truth",
100     xlabel="x",
101     ylabel="f(x)",
102     title="Truth_vs_NN_Approximation",
103     linewidth=3,
104     color=:steelblue,
105     legend=:topleft,
106     gridalpha=0.25,
107     left_margin=8mm,
108     right_margin=4mm,
109     top_margin=6mm,
110     bottom_margin=8mm
111 )
112 scatter!(
113     p1, xs, pred_vals,
114     label="NN_Model",
115     color=:tomato,
116     markersize=3,
117     markerstrokewidth=0,
118     alpha=0.85
119 )
120
121 p2 = plot(
122     1:length(losses), losses,
123     label="Training_Loss",
124     xlabel="Epoch",
125     ylabel="Loss",
126     title="Loss_vs_Number_of_Epochs",
127     linewidth=2.5,
128     marker=:circle,
129     markersize=3,
130     color=:darkgreen,

```

```

131     legend=:topright,
132     gridalpha=0.25,
133     left_margin=8mm,
134     right_margin=4mm,
135     top_margin=6mm,
136     bottom_margin=8mm
137 )
138
139 P = plot(
140     p1, p2,
141     layout=(1, 2),
142     size=(1300, 480),
143     dpi=150
144 )
145
146 savefig("NN_Fxn_Approx.png")

```

Problem 4:

The algorithms I chose to implement were Q-Learning and SARSA- λ . I chose to implement Q-Learning from scratch since it was essentially the same, if not easier than SARSA- λ . Q-Learning's main strength is its simplicity and doesn't require tracking eligibility traces like SARSA- λ . In addition, Q-Learning is off policy, meaning it learns the optimal policy regardless of the exploration policy. However, the use of max over the actions leads to maximization bias, causing at times slower learning. SARSA- λ on the other hand is on policy, which can be beneficial in some scenarios. Its main advantage is eligibility traces which allow for temporal credit assignment, meaning we can spread updates backward in time leading to faster convergence. Generally, Q-Learning has a lower sample complexity compared to SARSA- λ , and thus it often learns faster in terms of wall clock time. However, this is likely a small difference as they appear to learn just as quickly in figures 6 and 8. Despite this, I was able to achieve an average undiscounted reward above 5 with SARSA- λ and Q-Learning with little tuning required.

4.1 Figures

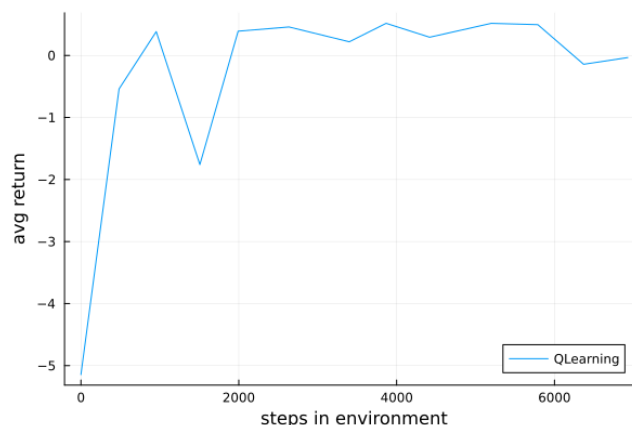


Figure 5: Q-Learning Average Return vs Steps in Environment

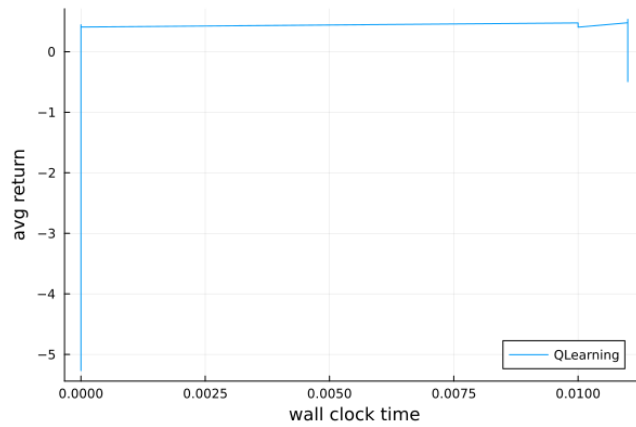
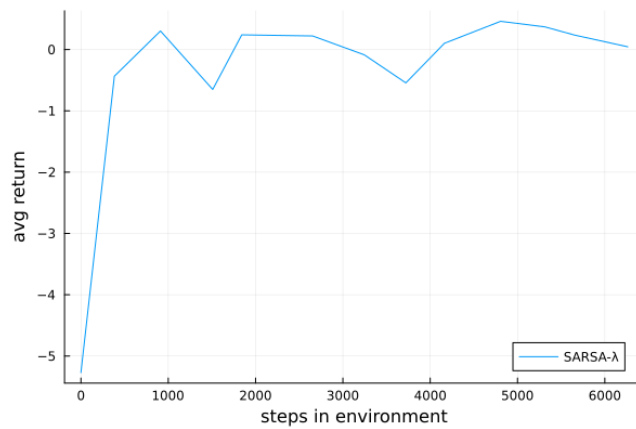
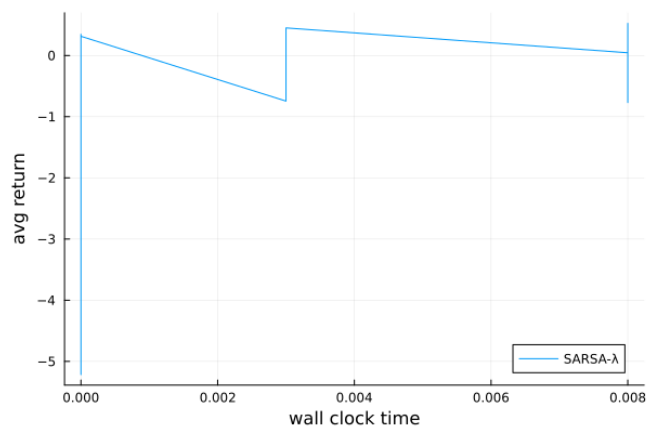


Figure 6: Q-Learning Average Return vs Wall Clock Time

Figure 7: SARSA- λ Average Return vs Steps in EnvironmentFigure 8: SARSA- λ Average Return vs Wall Clock Time

4.2 Q-Learning Code

```

1 using DMUStudent.HW4: HW4, gw, render
2 using CommonRLInterface: reset!, actions, act!, observe, terminated,
  observations
3 using Plots
4 using Statistics: mean
5
6 function QLearning(env; n_episodes=100000)
7     Q = Dict{(s, a) => 0.0 for s in observations(env), a in actions(env))
8     N = Dict{(s, a) => 0.0 for s in observations(env), a in actions(env))
9     episodes = []
10
11     for i in 1:n_episodes
12         reset!(env)
13         push!(episodes, Q_episode(Q, env, N;  $\epsilon$ =max(0.01, 1-i/n_episodes)))
14     end
15
16     return episodes
17 end
18
19 function policy(env, s,  $\epsilon$ , Q)
20     if rand() <  $\epsilon$ 
21         return rand(actions(env))
22     else
23         return argmax(a->Q[(s, a)], actions(env))
24     end
25 end
26
27 function evaluate(env, policy; n_episodes=1000, max_steps=1000,  $\gamma$ =1.0)
28     returns = Float64[]
29     for _ in 1:n_episodes
30         t = 0
31         r = 0.0
32         reset!(env)
33         s = observe(env)
34         while !terminated(env)
35             a = policy(s)
36             r +=  $\gamma^t$ *act!(env, a)
37             s = observe(env)
38             t += 1
39         end
40         push!(returns, r)
41     end
42     return returns
43 end
44
45 function Q_episode(Q, env, N;  $\epsilon$ =0.01,  $\gamma$ =0.99,  $\alpha$ =0.2)
46     start = time()
47
48     s = observe(env)
49     hist = [s]
50     rhist = []
51     while !terminated(env)

```

```

52     a = policy(env, s,  $\epsilon$ , Q)
53     r = act!(env, a)
54     sp = observe(env)
55
56     if terminated(env)
57         target = r
58     else
59         target = r +  $\gamma$  * maximum(ap -> Q[(sp, ap)], actions(env))
60     end
61     Q[(s,a)] +=  $\alpha$  * (target - Q[(s,a)])
62
63     s = sp
64
65     push!(hist, sp)
66     push!(rhist, r)
67 end
68
69 return (hist=hist, rhist=rhist, Q = copy(Q), time=time()-start)
70 end
71
72 episodes = QLearning(gw)
73
74
75 Q = episodes[end].Q
76 render(gw, policy = s -> argmax(a -> Q[(s,a)], actions(gw)))
77
78 function learning_curve_steps(episodes, env)
79     p = plot(xlabel="steps_in_environment", ylabel="avg_return")
80     n = 40
81     stop = 500
82     for (name, eps) in episodes
83         Q = Dict{Tuple{String, String}, Float64}()
84         for s in observations(env), a in actions(env)
85             Q[(s,a)] = 0.0
86         end
87         xs = [0]
88         ys = [mean(evaluate(env, s->argmax(a->Q[(s, a)], actions(env)))
89             for s in observations(env))]
90         for i in n:n:min(stop, length(eps))
91             newsteps = sum(length(ep.hist) for ep in eps[i-n+1:i])
92             push!(xs, last(xs) + newsteps)
93             Q = eps[i].Q
94             push!(ys, mean(evaluate(env, s->argmax(a->Q[(s, a)],
95                 actions(env)), n_episodes=1000)))
96         end
97         plot!(p, xs, ys, label=name)
98     end
99     p
100 end
101
102 function learning_curve_clock(episodes, env)
103     p = plot(xlabel="wall_clock_time", ylabel="avg_return")
104     n = 20
105     stop = 500
106     for (name, eps) in episodes

```

```

102     Q = Dict((s, a) => 0.0 for s in observations(env), a in
103             actions(env))
104     xs = [0.0]
105     ys = [mean(evaluate(env, s->argmax(a->Q[(s, a)], actions(env))
106             ))]
107     for i in n:n:min(stop, length(eps))
108         newtime = sum(ep.time for ep in eps[i-n+1:i])
109         push!(xs, last(xs) + newtime)
110         Q = eps[i].Q
111         push!(ys, mean(evaluate(env, s->argmax(a->Q[(s, a)],
112             actions(env)), n_episodes=1000)))
113     end
114     plot!(p, xs, ys, label=name)
115 end
116 # Associate episodes with a name
117 Qepisodes = [("QLearning", episodes)]
118 learning_curve_steps(Qepisodes, gw)
119 savefig("Q_Steps.png")
120
121 learning_curve_clock(Qepisodes, gw)
122 savefig("Q_Clock.png")
123
124 Q_final = episodes[end].Q
125 policy_final = s -> argmax(a -> Q_final[(s, a)], actions(gw))
126
127 returns_undiscounted = evaluate(gw, policy_final; n_episodes=1000,  $\gamma=1.0$ )
128 avg_undiscounted = mean(returns_undiscounted)
129
130 println("Q-Learning Performance Check:")
131 println("=====")
132 println("Average undiscounted cumulative reward: $avg_undiscounted")
133 println("Requirement: > 5")
134 println("Success: $(avg_undiscounted > 5 ? "YES" : "NO")")

```

4.3 SARSA- λ Code

```

1     using DMUStudent.HW4: HW4, gw, render
2     using CommonRLInterface: reset!, actions, act!, observe, terminated,
3         observations
4     using Plots
5     using Statistics: mean
6     function sarsa_lambda_episode!(Q, env;  $\epsilon=0.1$ ,  $\gamma=0.99$ ,  $\alpha=0.2$ ,  $\lambda=0.99$ )
7         start = time()
8
9         s = observe(env)
10        a = rand(actions(env))
11        r = act!(env, a)
12        sp = observe(env)
13        hist = [s]

```

```

14     N = Dict{(s, a) => 0.0)
15
16     function policy(s)
17         if rand() <  $\epsilon$ 
18             return rand(actions(env))
19         end
20         return argmax(a -> Q[(s,a)], actions(env))
21     end
22
23
24     while !terminated(env)
25         ap = policy(sp)
26
27         N[(s, a)] = get(N, (s, a), 0.0) + 1
28
29          $\delta$  = r +  $\gamma$ *Q[(sp, ap)] - Q[(s, a)]
30
31         for ((s, a), n) in N
32             Q[(s, a)] +=  $\alpha$ * $\delta$ *n
33             N[(s, a)] *=  $\gamma$ * $\lambda$ 
34         end
35
36         s = sp
37         a = ap
38         r = act!(env, a)
39         sp = observe(env)
40         push!(hist, sp)
41     end
42
43     N[(s, a)] = get(N, (s, a), 0.0) + 1
44      $\delta$  = r - Q[(s, a)]
45
46     for ((s, a), n) in N
47         Q[(s, a)] +=  $\alpha$ * $\delta$ *n
48         N[(s, a)] *=  $\gamma$ * $\lambda$ 
49     end
50
51     return (hist=hist, Q = copy(Q), time=time()-start)
52 end
53 function sarsa_lambda!(env; n_episodes=100, kwargs...)
54     Q = Dict{(s, a) => 0.0 for s in observations(env), a in actions(env))
55     episodes = []
56
57     for i in 1:n_episodes
58         reset!(env)
59         push!(episodes, sarsa_lambda_episode!(Q, env;
60                                      $\epsilon$ =max(0.01, 1-i/n_episodes),
61                                     kwargs...))
62     end
63
64     return episodes
65 end
66 function evaluate(env, policy; n_episodes=50_000, max_steps=1000,  $\gamma$ =1.0)
67     returns = Float64[]

```

```

68     for _ in 1:n_episodes
69         t = 0
70         r = 0.0
71         reset!(env)
72         s = observe(env)
73         while !terminated(env)
74             a = policy(s)
75             r +=  $\gamma^t$ *act!(env, a)
76             s = observe(env)
77             t += 1
78         end
79         push!(returns, r)
80     end
81     return returns
82 end
83
84 lambda_episodes = sarsa_lambda!(gw, n_episodes=100_000,  $\alpha$ =0.05,  $\lambda$ =0.9);
85 episodes = Dict("SARSA- $\lambda$ "=>lambda_episodes)
86
87
88 Q = lambda_episodes[end].Q
89 render(gw, policy = s -> argmax(a -> Q[(s,a)], actions(gw)))
90
91 function learning_curve_steps(episodes, env)
92     p = plot(xlabel="steps_in_environment", ylabel="avg_return")
93     n = 40
94     stop = 500
95     for (name, eps) in episodes
96
97         Q = Dict{(s, a) => 0.0 for s in observations(env), a in
98             actions(env))
99         xs = [0]
100         ys = [mean(evaluate(env, s->argmax(a->Q[(s, a)], actions(env))
101             ))]
102         for i in n:n:min(stop, length(eps))
103             newsteps = sum(length(ep.hist) for ep in eps[i-n+1:i])
104             push!(xs, last(xs) + newsteps)
105             Q = eps[i].Q
106             push!(ys, mean(evaluate(env, s->argmax(a->Q[(s, a)],
107                 actions(env)), n_episodes=1000)))
108         end
109         plot!(p, xs, ys, label=name)
110     end
111     p
112 end
113 learning_curve_steps(episodes, gw)
114 savefig("SARSA_Steps.png")
115
116 function learning_curve_clock(episodes, env)
117     p = plot(xlabel="wall_clock_time", ylabel="avg_return")
118     n = 20
119     stop = 500
120     for (name, eps) in episodes

```

```

118     Q = Dict((s, a) => 0.0 for s in observations(env), a in
119             actions(env))
120     xs = [0.0]
121     ys = [mean(evaluate(env, s->argmax(a->Q[(s, a)], actions(env))
122             ))]
123     for i in n:n:min(stop, length(eps))
124         newtime = sum(ep.time for ep in eps[i-n+1:i])
125         push!(xs, last(xs) + newtime)
126         Q = eps[i].Q
127         push!(ys, mean(evaluate(env, s->argmax(a->Q[(s, a)],
128             actions(env)), n_episodes=1000)))
129     end
130     plot!(p, xs, ys, label=name)
131 end
132 p
133 learning_curve_clock(episodes, gw)
134 savefig("SARSA_Clock.png")
135
136 Q_final = lambda_episodes[end].Q
137 policy_final = s -> argmax(a -> Q_final[(s, a)], actions(gw))
138
139 returns_undiscounted = evaluate(gw, policy_final; n_episodes=1000,  $\gamma=1.0$ )
140 avg_undiscounted = mean(returns_undiscounted)
141
142 println("SARSA-Lambda_Performance_Check:")
143 println("=====")
144 println("Average_undiscounted_cumulative_reward:_$avg_undiscounted")
145 println("Requirement:>_5")
146 println("Success:_(avg_undiscounted>_5?"YES":_5?"NO"))

```